ARM PrimeCell Driver

test v1.5 (System Configuration)

API Specification

PL000-ISPC-1003-D

Open Access

Proprietary notice

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	D
Primecell driver identifier	test
Code Version	1.5
Primecell identifier(s)	PL000

Contents

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A		First issue of API
B	15/11/00	Revised document to match desired template format

2 SUMMARY

The system configuration test module builds a test application to test system integration.

This application will run the self-test modules for the drivers for each peripheral present in the system, and will return an OR'd result of these to indicate whether the system as a whole is correctly implemented.

3 FEATURES

The main features supported by the system configuration test module are.

- Public test routines available for use in separate applications or built into the test application.
- Devices to test easily selected by extending the macro **`TEST_ALL_MODULES`** (defined in `apos\applatm.h`), so allowing a subset of the full tests easily to be selected.
- Bootstrap code and vector table supplied for simple setup of ARM
- Code supplied to enable caches and MMU/MPU

3.1 Specifying the modules to test

3.1.1 TEST_ALL_MODULES

The set of PrimeCells to be tested is specified by a macro TEST_ALL_MODULES defined in **`applatfm.h`**. This is defined as a list of peripherals as in Example 1.

Example 1 - Determining modules to test

```
#define TEST_ALL_MODULES \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,0), apOS_SYSTEM_BASE_TIMER0,   apOS_INT_TIMER0); \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,1), apOS_SYSTEM_BASE_TIMER1,   apOS_INT_TIMER1); \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,2), apOS_SYSTEM_BASE_TIMER2,   apOS_INT_TIMER2); \
    TEST_MODULE(KMI,   0, apOS_INSTANCE(KMI,0),   apOS_SYSTEM_BASE_KMI0,     apOS_INT_KMI0); \
    TEST_MODULE(KMI,   0, apOS_INSTANCE(KMI,1),   apOS_SYSTEM_BASE_KMI1,     apOS_INT_KMI1); \
    TEST_MODULE(UART,  0, apOS_INSTANCE(UART,0),  apOS_SYSTEM_BASE_UART0,    apOS_INT_UART0); \
    TEST_MODULE(UART,  0, apOS_INSTANCE(UART,1),  apOS_SYSTEM_BASE_UART1,    apOS_INT_UART1); \
    TEST_MODULE(RTC,  30, apOS_INSTANCE(RTC,0),   apOS_SYSTEM_BASE_RTC,      apOS_INT_RTC);
```

Each line of the TEST_ALL_MODULES macro except the last must have a terminating backslash (escape) character, to ensure that they form part of the same macro. Care must be taken in commenting out sections of the macro. The pre-processor will remove commented sections, thereby possibly losing the backslash characters. Example 2, in which the user has attempted to remove the KMI tests by adding C++ style comment markers, will not compile correctly.

Example 2 – Incorrect manipulation of the TEST_ALL_MODULES macro

```
#define TEST_ALL_MODULES \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,0), apOS_SYSTEM_BASE_TIMER0,   apOS_INT_TIMER0); \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,1), apOS_SYSTEM_BASE_TIMER1,   apOS_INT_TIMER1); \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,2), apOS_SYSTEM_BASE_TIMER2,   apOS_INT_TIMER2); \
//  TEST_MODULE(KMI,   0, apOS_INSTANCE(KMI,0),   apOS_SYSTEM_BASE_KMI0,     apOS_INT_KMI0); \
//  TEST_MODULE(KMI,   0, apOS_INSTANCE(KMI,1),   apOS_SYSTEM_BASE_KMI1,     apOS_INT_KMI1); \
    TEST_MODULE(UART,  0, apOS_INSTANCE(UART,0),  apOS_SYSTEM_BASE_UART0,    apOS_INT_UART0); \
    TEST_MODULE(UART,  0, apOS_INSTANCE(UART,1),  apOS_SYSTEM_BASE_UART1,    apOS_INT_UART1); \
    TEST_MODULE(RTC,  30, apOS_INSTANCE(RTC,0),   apOS_SYSTEM_BASE_RTC,      apOS_INT_RTC);
```

3.1.2 TEST_MODULE

The TEST_MODULE macro takes five parameters as follows:

- Name of driver to test (e.g. UART)
- The PrimeCell number of the driver (e.g. 11 for the UART PL011). Many PrimeCells support identification registers and this number will be checked against these registers. If the PrimeCell does not support this, the value 0 should be specified.
- The enumeration of the PrimeCell to test, specified using the apOS_INSTANCE() macro
- The base address of the PrimeCell being tested as an enumeration
- The interrupt source assigned to the PrimeCell being tested as an enumeration (see §3.1.4,§3.1.5)

3.1.3 apOS_INSTANCE

The apOS_INSTANCE macro is used to specify the PrimeCell to test. It takes two parameters as follows:

- Name of driver to test (e.g. UART)
- Number of that PrimeCell to test

In the normal build, this macro will combine the two arguments ***MOD*** and ***N*** to produce an enumerator apOS_***MOD_N***.

If the constant apOS_NO_STATIC_STATE is declared, an area of memory is assigned on the heap for the test using malloc.

3.1.4 TEST_NOINT

If the PrimeCell has no assigned interrupts, the TEST_NOINT constant should be passed as the last parameter to TEST_MODULE.

3.1.5 TEST_LIST

If the PrimeCell has multiple assigned interrupts, the TEST_LIST macro is used to specify the last parameter to TEST_MODULE.

TEST_LIST takes two parameters, each of which may be an interrupt source assigned to the PrimeCell being tested as an enumeration or another TEST_LIST macro. Example 3 shows how a test would be specified for a hypothetical PrimeCell PL123 of type MOD, with three interrupt sources.

Example 3 - Specifying three interrupt sources

```
#define TEST_ALL_MODULES \
 TEST_MODULE(MOD, 123, apOS_MOD_0, apOS_SYSTEM_BASE_MOD, \
                  TEST_LIST(apOS_INT_MOD1, \
                            TEST_LIST(apOS_INT_MOD2,apOS_INT_MOD3) \
                           ) \
                  );
```

4 DETAILS

- Peak performance obtained in tests: N/A
- ROM size (RO code + RO data) (release build): 2104 Bytes
- RAM size (ZI data) per instance (release build): 16 Bytes
- Does this driver support both endian-nesses?[1] YES
- Does this driver support Thumb?[2] YES

1 To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

2 The driver may include ARM code, but this must interwork with a Thumb build.

5 RELEASED FILES

PL000-ISPC-1003-D	This API document
aptest/aptest.h	Public header file
aptest/test.h	Private header file
aptest/test.c	Source code for test routines
aptest/main.c	Source code for main.c application code
aptest/boot.s	Bootstrap code
aptest/vectors.s	Code for exception vectors
aptest/pagetab.s	Space for page table – only required for cores with MMU

6 PUBLIC API

6.1 Type Definitions

6.1.1 apTEST_eFormat

Allowed formats for test diagnostics

Declaration

```
typedef enum apTEST_eFormat
```

Elements

apTEST_FORMAT_LONG_DEC	long integer in decimal
apTEST_FORMAT_LONG_HEX	long integer in hex
apTEST_FORMAT_NO_NUMBER	do not report parameter

6.1.2 apTEST_eResult

Test result states

Declaration

```
typedef enum apTEST_eResult
```

Elements

apTEST_FAIL	failure of test
apTEST_PASS	test passes

6.1.3 apTEST_rRoutine

Format for a test routine

Declaration

```
typedef apTEST_eResult apTEST_rRoutine(UWORD32 oId, UWORD32 RegBase,
                  CONST apOS_INT_oInterruptSource * CONST pInt)
```

Remarks

oId	the identifier for the peripheral instance to use
RegBase	base address for registers.
pInt	pointer to a list of identifiers for interrupts used.

6.2 External Functions

6.2.1 apTEST_BufferedPrint

This routine writes a message, preceded by any buffered carriage return. Carriage returns are buffered to allow messages to be joined.

Declaration

```
PUBLIC void apTEST_BufferedPrint(char * CONST pMessage)
```

Parameters

pMessage	a string message to be displayed.

Return Value

none

6.2.2 apTEST_EnableCaches

This routine enables system caches.

Declaration

```
PUBLIC void apTEST_EnableCaches(void)
```

Return Value

none

Remarks

If the code is compiled under ADS as ARM720,ARM740,ARM920 or ARM940 the caches will be enabled. This relies on the preprocessor constant __TARGET_CPU_ARMnnnT being defined to indicate the desired core type.

6.2.3 apTEST_EnableInterrupts

This routine enables the necessary interrupt controllers.

Declaration

```
PUBLIC void apTEST_EnableInterrupts(void)
```

Return Value

none

Remarks

This code will enable an INT and/or VIC interrupt controller according to the setting of apINT_VERSION in apos/apintcfg.h.

6.2.4 apTEST_EnableTimers

This routine enables the necessary system timers.

Declaration

```
PUBLIC void apTEST_EnableTimers(void)
```

Return Value

none

Remarks

If the code is compiled with apOS_NO_STATIC_STATE = FALSE and apOS_PLATFORM_HAS_TIMERS = TRUE, this function will initialise one system timer (apOS_TIMER_0) for use with timeouts.

6.2.5 apTEST_ImplementBasicTests

This routine performs some basic system tests.

Declaration

```
PUBLIC apTEST_eResult apTEST_ImplementBasicTests(void)
```

Return Value

- apTEST_PASS if test is successful
- apTEST_FAIL if test is unsuccessful

Remarks

Currently this performs word, halfword and byte write/read cycles. Further tests can be added.

6.2.6 apTEST_Print

This routine prints a test message and is used by the module test routines for reporting

Declaration

```
PUBLIC void apTEST_Print(apTEST_eResult eItemResult, char * CONST pTestName)
```

Parameters

eItemResult	specifies whether the test passed (in which case the message is informational only) or failed. Displaying a failure message does not terminate the test.
pTestName	a string message to be displayed.

Return Value

none

Remarks

Where possible in a debug environment, this routine will print the message, which is the name of the test being undertaken, followed by 'Pass' or 'Fail' according to the result parameter.

This routine should be customised if a modified data output method is required. In extreme cases, the result might be implemented only by displaying a red or green light according to the Result parameter

6.2.7 apTEST_Remove_CR

This routine suppresses the carriage return generated by the previous call to a test output routine

Declaration

```
PUBLIC void apTEST_Remove_CR(void)
```

Return Value

none

Remarks

Discards the buffered carriage return in memory

6.2.8 apTEST_Report

This routine prints a diagnostic message followed by a value.

Declaration

```
PUBLIC void apTEST_Report(char * CONST pTestName,
                          UWORD32 DataValue,
                          apTEST_eFormat eDataFormat)
```

Parameters

pTestName	a string message to be displayed.
DataValue	the variable to be displayed
eDataFormat	the format to be used

Return Value

none

Remarks

The value may be one of several data types which should be specified as one of the constants TEST_FORMAT_*. If the apTEST_FORMAT_NO_NUMBER parameter is specified, no terminating carriage return is generated.

6.2.9 apTEST_TimeGet

This routine returns a clock value in ms. It may be used for timing.

Declaration

```
PUBLIC UWORD32 apTEST_TimeGet(void)
```

Return Value

Time in milliseconds

Remarks

Implemented using clock() unless apOS_CONFIG_USE_NO_CLIB is defined non-zero, in which case zero is always returned.

6.2.10 apTEST_WaitKey

This routine prints a text message and then waits for a keypress from the user.

Declaration

```
PUBLIC void apTEST_WaitKey(char * CONST pMessage, UWORD32 WaitTime)
```

Parameters

pMessage a string message to be displayed.

Return Value

none

6.3 External Variables

6.3.1 apTEST_TestInProgress

When testing with a timeout handler, apTEST_TestInProgress**{**xe "apTEST_TestInProgress"**}** is set to FALSE by the ISR which is being tested. This indicates that the test is complete

Declaration

```
extern volatile BOOL apTEST_TestInProgress
```

6.3.2 apTEST_Timeout

When testing with a timeout handler, apTEST_Timeout**{**xe "apTEST_Timeout"**}** is set to TRUE by the timeout handler. This indicates that the test timed out

Declaration

```
extern volatile BOOL apTEST_Timeout
```

6.4 Macros

6.4.1 TEST_LIST

This macro is used to specify a set of interrupt sources to be passed to TEST_MODULE{xe "TEST_MODULE"}

Declaration

```
#define TEST_LIST(_int1,_int2) (CONST apOS_INT_oInterruptSource) (_int1),(CONST
apOS_INT_oInterruptSource) (_int2)
```

6.4.2 TEST_MODULE

Declaration

```
#define TEST_MODULE(_mod,_id,_inst,_base,_int) \
       {extern apTEST_rRoutine ap ## _mod ## _SelfTest;\
        __weak UWORD32 ap ## _mod ##_##StateSizeGet(void);\
        CONST apOS_INT_oInterruptSource IntArray[]={_int};\
        TestFlag= (apTEST_eResult) (TestFlag & TEST_Module( # _mod,ap ## _mod ##
_SelfTest,_id,(UWORD32)_inst,_base,&IntArray[0]));}
```

Remarks

_mod	name of module (e.g. MMC).
_id	Primecell number (e.g. 180)
_inst	the instance of the driver to be initialised (e.g. apOS_MOD_0 for the first instance of module MOD)
_base	base address of registers (e.g. 0x1234578)
_int	set of identifiers of interrupts. There may be any number of these, but they must be specified using the TEST_LIST{xe "TEST_LIST"} macro e.g. TEST_LIST{xe "TEST_LIST"}(LIST(apOS_INT_ID1,apOS_INT_ID2),apOS_INT_ID3)

6.4.3 TEST_MODULE_P

This macro is used to specify a test on a module. TEST_MODULE{xe "TEST_MODULE"} is preferred, but this is maintained for backwards compatibility. It performs the standard test, but with peripheral instance always set to zero.

Declaration

```
#define TEST_MODULE_P(_mod,_id,_base,_int) TEST_MODULE(_mod,_id,0,_base,_int)
```

Remarks

_mod	name of module (e.g. MMC).
_id	Primecell number (e.g. 180)
_base	base address of registers (e.g. 0x1234578)
_int	set of identifiers of interrupts. There may be any number of these, but they must be specified using the TEST_LIST{xe "TEST_LIST"} macro e.g. TEST_LIST{xe "TEST_LIST"}(LIST(apOS_INT_ID1,apOS_INT_ID2),apOS_INT_ID3)

6.4.4 TEST_NOINT

This macro is used for clarity to specify that there are no interrupts used

Declaration

```
#define TEST_NOINT (CONST apOS_INT_oInterruptSource) 0
```

6.4.5 apOS_INSTANCE

This macro allows the PrimeCell instance to be switched according to the state of apOS_NO_STATIC_STATE.

- If apOS_NO_STATIC_STATE=FALSE, apOS_INSTANCE(MOD,0) will resolve to enumeration apOS_MOD_0.
- If apOS_NO_STATIC_STATE=TRUE, apOS_INSTANCE(MOD,0) will resolve to a malloc of the appropriate size, using apMOD_StateSizeGet()

Declaration

```
#define apOS_INSTANCE(_mod,_inst) malloc((HWD16)ap ## _mod ##_## StateSizeGet())
```

6.4.6 apTEST_END_ON_FIRST_FAILURE

apTEST_END_ON_FIRST_FAILURE**{**xe "apTEST_END_ON_FIRST_FAILURE"**}** - if TRUE, the test will terminate when any module returns a failure result.

Declaration

```
#define apTEST_END_ON_FIRST_FAILURE       FALSE
```

6.4.7 apTEST_INTERACTIVE

apTEST_INTERACTIVE**{**xe "apTEST_INTERACTIVE"**}** - if TRUE, the user is present during testing and can press a key to continue a test after a pause

Declaration

```
#define apTEST_INTERACTIVE            FALSE
```

6.4.8 apTEST_TIMEOUT

Macro for testing peripherals with a timeout handler

Declaration

```
#define apTEST_TIMEOUT(__testFunc, __oId, __ntimer, __period, __msg, __result)        \
    {extern void apTEST_GenericTimeout(UWORD32);                                        \
    apTEST_TestInProgress = TRUE;                                                      \
    apTEST_Timeout = FALSE;                                                            \
    apTEST_Report("(timeout after ", (__period), apTEST_FORMAT_LONG_DEC);              \
    apTEST_Remove_CR();                                                                \
    apTEST_Report(" seconds)\n", 0, apTEST_FORMAT_NO_NUMBER);                          \
    apTIMER_IntervalSet((__ntimer), &apTEST_GenericTimeout, 0, (__period)*1000000, 1); \
    (__testFunc)(__oId);                                                               \
    while(apTEST_TestInProgress);                                                      \
    apTIMER_Disable(__ntimer);                                                         \
    if (apTEST_Timeout)                                                                \
    {                                                                                  \
        apTEST_Print((__result), (__msg));                                             \
        return (__result);                                                             \
    }}
```

Remarks

apTEST_TIMEOUT**{**xe "apTEST_TIMEOUT"**}**(__testFunc, __oId, __ntimer, __period, __msg, __result)

- __testFunc is the function to set up the test (takes one parameter - the peripheral identifier)
- __oId is the peripheral identifier (leave blank if __testFunc takes no parameter)
- __ntimer is the Id of the timer to use
- __period is the timeout period in seconds
- __msg is the message to return on timeout
- __result is the apTEST_eResult**{**xe "apTEST_eResult"**}** to return on timeout

__testFunc should set up the registers and return so that the test is controlled by interrupts. If it is necessary to keep program flow inside __testFunc, then it can contain a while(apTEST_TestInProgress**{**xe "apTEST_TestInProgress"**}**) loop. When the test is over, the ISR should set apTEST_TestInProgress**{**xe "apTEST_TestInProgress"**}** to false.

6.4.9 apTEST_VERBOSE

apTEST_VERBOSE**{**xe "apTEST_VERBOSE"**}** - if TRUE, any messages will be shown, even if they do not terminate the test; behaviour is modified by configuration flag apOS_CONFIG_USE_NO_CLIB, which, if zero or undefined, means there is no C Library support, and hence no printing available

Declaration

```
#define apTEST_VERBOSE                  FALSE
```

Remarks